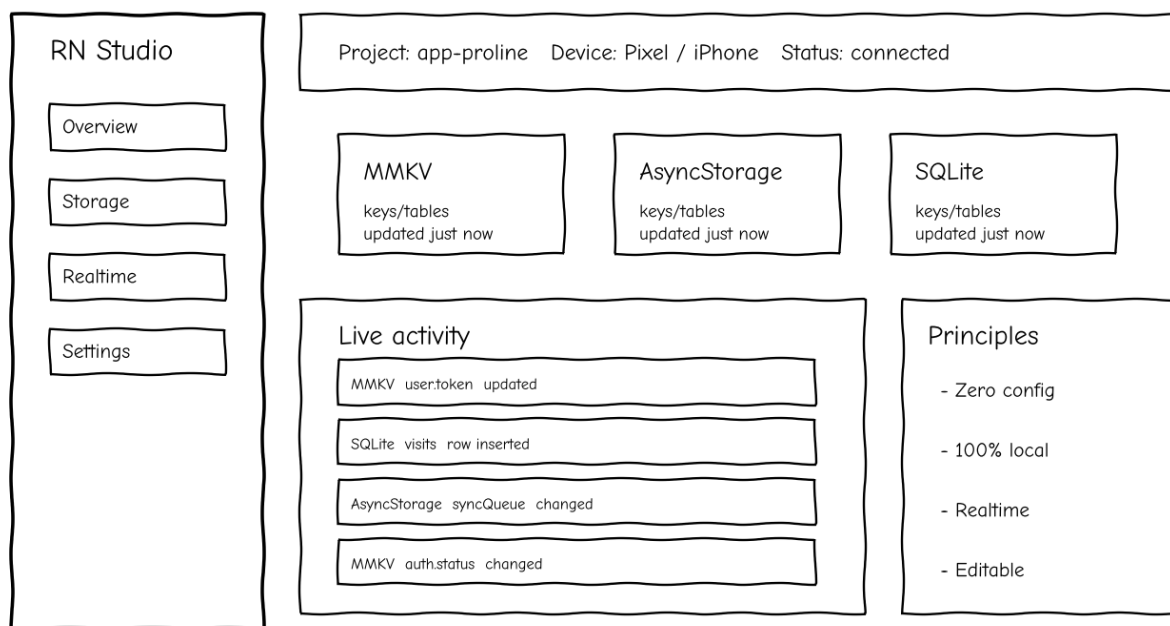


RN Studio

MVP — Documento de Produto, Escopo e Arquitetura Técnica

Uma proposta para construir um inspetor universal de dados locais para aplicações React Native.

Low-fi prototype — Dashboard



Problema	Inspecionar dados locais em apps React Native ainda é fragmentado, manual e pouco confiável.
Produto	Ferramenta desktop 100% local para visualizar, editar e acompanhar em tempo real MMKV, AsyncStorage
Princípios	Zero configuration • 100% local • Realtime • Simples de evoluir

1. A dor

Hoje, inspecionar os dados de uma aplicação React Native exige uma combinação pouco elegante de ferramentas: Flipper, plugins inconsistentes, adb shell, console.log, consultas SQL manuais, navegação de arquivos e inspeções ad hoc feitas dentro do próprio app. O desenvolvedor perde tempo, aumenta a chance de erro e não possui um único lugar para entender o estado real da aplicação.

Principais dores observadas

- Não existe uma experiência única e coesa para dados locais no ecossistema React Native.
- MMKV, AsyncStorage e SQLite costumam ser inspecionados de maneiras completamente diferentes.
- Editar dados durante o desenvolvimento ainda é uma experiência frágil e pouco visual.
- Ferramentas atuais raramente oferecem um bom modelo de realtime entre o app e a interface de inspeção.
- Boa parte do trabalho de debugging ainda acontece por tentativas, logs e scripts improvisados.

Hipótese de valor

Se entregarmos uma ferramenta desktop, local e simples, capaz de detectar automaticamente os principais storages do projeto e expor uma interface consistente para leitura, edição e acompanhamento em tempo real, a produtividade do time aumenta, o debugging fica mais previsível e a adoção tem potencial para ser muito alta.

2. O produto

RN Studio é um inspetor de dados locais para React Native. No MVP, ele foca em um problema muito bem definido: permitir que o desenvolvedor visualize, pesquise, edite e acompanhe em tempo real os dados armazenados em MMKV, AsyncStorage e SQLite.

Promessa central do MVP

- Instalou, abriu, funcionou.
- Nenhum dado sai da máquina do desenvolvedor.
- Mudanças feitas no Studio devem refletir imediatamente no storage da aplicação.
- Quando a tecnologia suportar reatividade, o próprio app também reflete a mudança automaticamente.
- A interface é consistente independentemente do provider.

Escopo do MVP

Em escopo	MMKV, AsyncStorage, SQLite (implementação inicial via Expo SQLite), leitura, busca, criação
Fora do MVP	Network, logs, localização, notificações, performance avançada, autenticação, nuvem, colaboração

3. Princípios de experiência

- **Zero configuration por padrão:** um pacote principal, um comando principal. Detectar automaticamente o máximo possível.
- **100% local e privado:** sem cloud, sem login, sem telemetria obrigatória. Dados da aplicação nunca saem da máquina.
- **Realtime de verdade:** o Studio deve reagir a mudanças quase instantaneamente e dar feedback visual sobre o que mudou.
- **Simplicidade radical:** interface óbvia até para um dev iniciante. Nada de configuração excessiva ou steps técnicos longos.
- **Arquitetura evolutiva:** o MVP começa pequeno, mas já nasce pronto para receber novos providers no futuro.

Fluxo ideal do usuário

```
npm install --save-dev rn-studio  
npx rn-studio
```

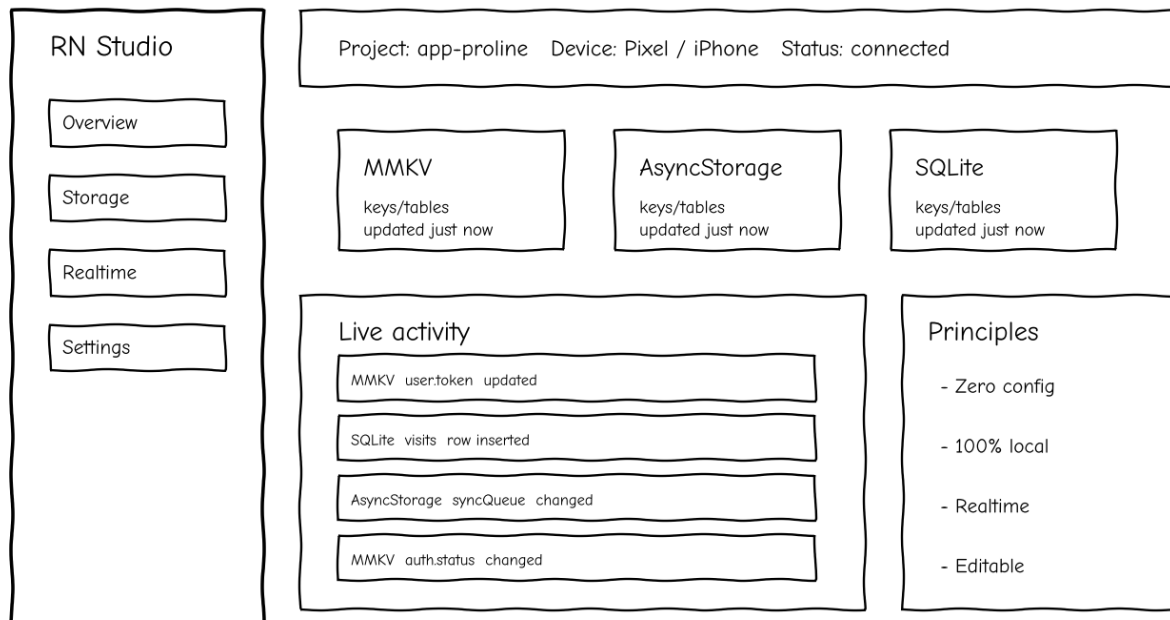
Resultado esperado:

- A CLI identifica React Native/Expo e as dependências relevantes.
- O serviço local é iniciado.
- O runtime do app se conecta ao serviço local.
- A aplicação desktop é aberta automaticamente.
- O usuário já enxerga MMKV, AsyncStorage e SQLite em uma interface única.

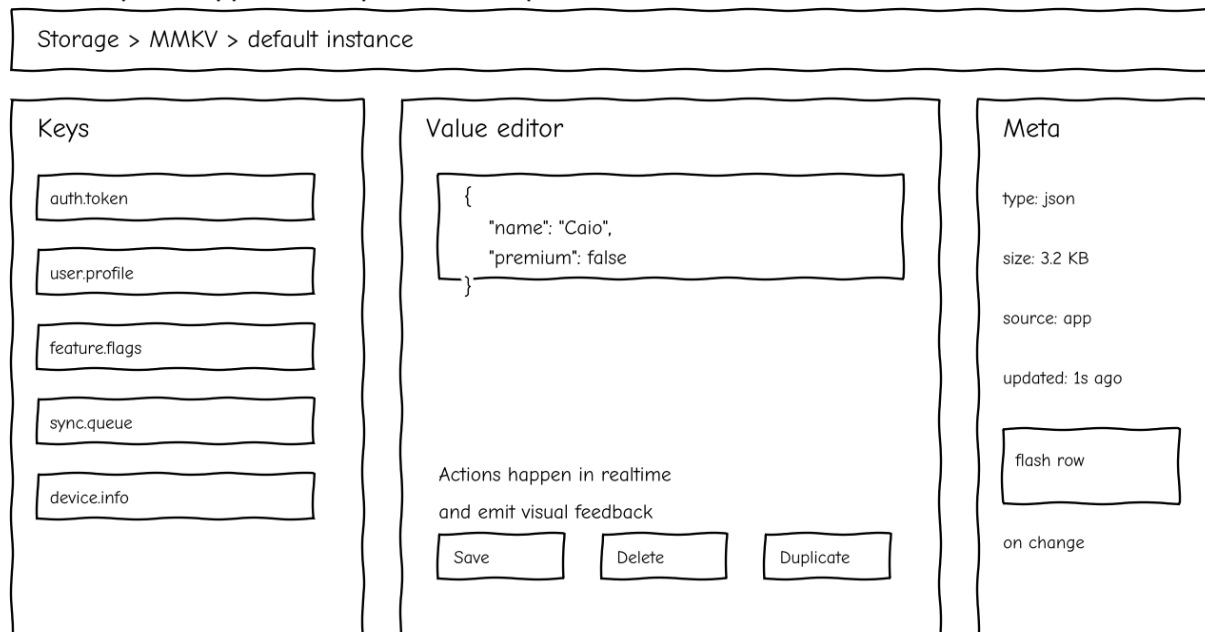
4. Protótipos low-fi

A seguir, alguns rabiscos conceituais do produto. O objetivo aqui não é design final, mas sim comunicar estrutura, fluxo e proposta.

Low-fi prototype — Dashboard



Low-fi prototype — Key-value inspector



Low-fi prototype — SQLite browser

Storage > SQLite > proline.db > visits

Tables

visits

tasks

photos

users

queue

id	status	startedAt	finishedAt	pdv
1	done	08:00	08:37	Carrefour
2	pending	-	-	Pague Menos
3	done	09:05	09:44	Atacadao

SQL console

SELECT * FROM visits WHERE status = "pending";

Run

5. Funcionalidades do MVP

Capacidades comuns que a experiência deve oferecer, independentemente do provider:

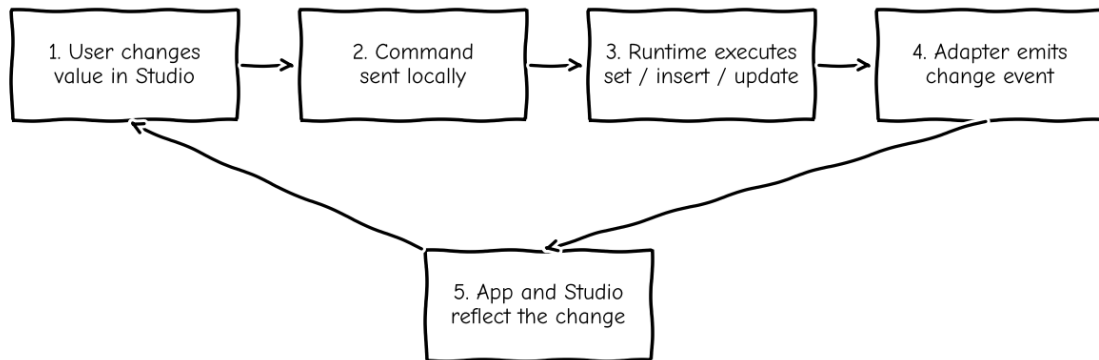
- Listar instâncias detectadas.
- Listar chaves, tabelas e entidades.
- Pesquisar rapidamente por nomes, valores e registros.
- Abrir um valor/registro em modo de leitura e edição.
- Criar novos valores ou linhas.
- Excluir valores ou registros com confirmação.
- Registrar mudanças em tempo real e destacar visualmente o que mudou.
- Diferenciar origem da mudança: app ou Studio.

Provider	Capacidades do MVP
MMKV	Listar chaves, ler tipos, editar valores, criar chaves, remover, renomear (copiar + remover), ouvir alterações
AsyncStorage	Listar chaves, ler valores, editar, criar, remover, acompanhar mudanças via instrumentação + fallback
SQLite	Listar bancos e tabelas, paginar linhas, editar células, inserir e remover registros, executar SQL manual

6. Realtime no MVP

O objetivo é chegar o mais próximo possível de uma sensação “tipo Firebase”: uma mudança acontece e a interface responde imediatamente. No entanto, é importante ser tecnicamente honesto: garantir o realtime dentro do Studio é totalmente viável; garantir que qualquer tela do app reaja automaticamente depende da forma como essa tela consome os dados.

Realtime loop



Target experience: Firebase-like feedback, but fully local.

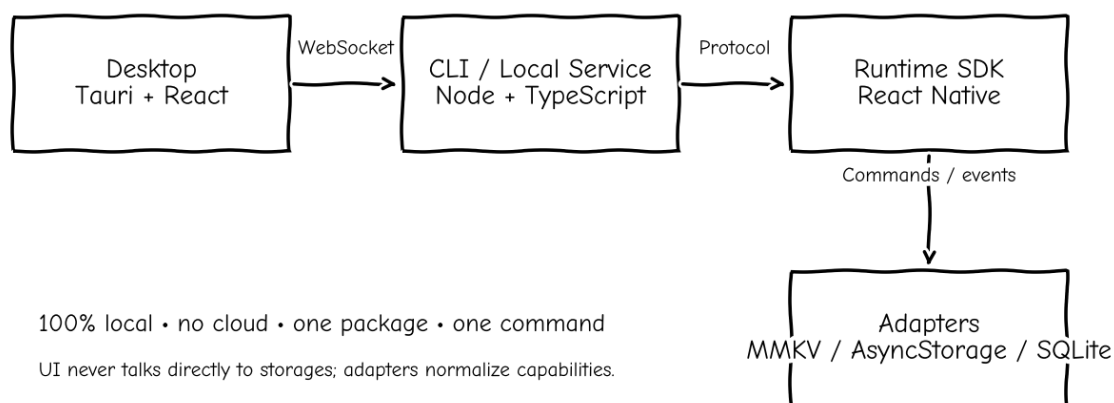
Nível de viabilidade por provider

Provider	Studio detecta mudanças locais	Studio altera o dado	App reage automaticamente
MMKV	Sim, via listeners nativos.	Sim.	Em geral, sim, quando o app usa hooks/listeners adequados.
AsyncStorage	Parcial: instrumentação + fallback.	Sim.	Não universal; depende de como o app recarrega esses dados.
SQLite	Sim, via change listener + reconstrução.	Sim.	Depende da camada de consumo (live query, invalidation, etc.).

7. Arquitetura proposta

A arquitetura recomendada para o MVP é um monólito modular em monorepo, com separação clara entre interface, serviço local, runtime React Native e adapters. O sistema deve ser simples de manter, mas preparado para evolução.

Architecture overview



Decisões principais

- **Tauri** para o desktop: mais leve que Electron e alinhado com a proposta de ferramenta rápida e local.
- **Node.js + TypeScript** para a CLI e o serviço local: excelente DX, facilidade de integração com o ecossistema RN e debugging simples.
- **Runtime SDK em TypeScript** para o app React Native, com suporte a módulos nativos/JSI apenas quando realmente necessário.
- **Adapters internos** para cada provider, mantendo a UI desacoplada das implementações reais.
- **Protocol em commands/events** para garantir comunicação robusta e testável entre as camadas.

8. Stack técnica

Camada	Stack recomendada	Motivo
Desktop	Tauri + React + Vite + TypeScript	Leve, moderna e suficientemente poderosa para o escopo.
Estado da UI	TanStack Query + Zustand	Query para dados remotos/runtime; Zustand para estado local de int
Edição avançada	Monaco Editor	Ótimo para JSON e SQL, reforça a sensação de ferramenta técnica.
CLI / Serviço local	Node.js + TypeScript	Descoberta do projeto, sessions, WebSocket e orquestração.
Comunicação	WebSocket local + protocolo JSON com Zuplo	Tempo real, simplicidade e contrato validado.
Runtime	TypeScript + RN runtime hooks	Baixo atrito para instalar e evoluir.
Monorepo	pnpm workspaces	Dependências mais corretas, workspaces claros e ótimo encaixe pa
Testes	Vitest + Playwright + app playground	Cobertura unitária, contracts, integração e E2E.

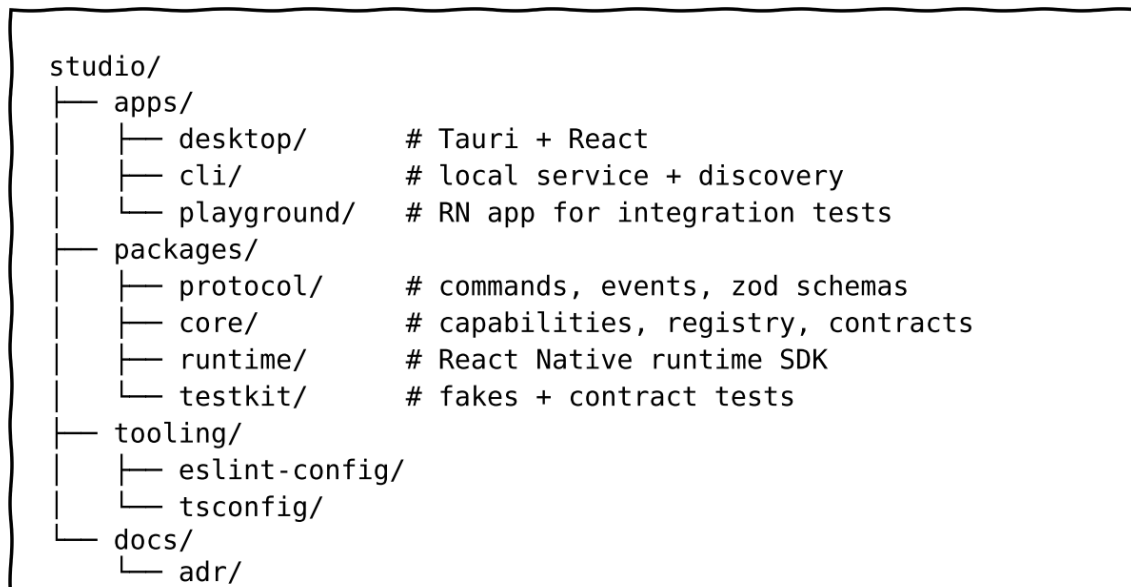
Por que pnpm?

- Excelente encaixe com monorepo e múltiplos pacotes internos.
- Ajuda a evitar dependências fantasmas entre packages.
- Permite usar workspace : * para garantir linkagem correta entre pacotes locais.
- Entrega uma base mais segura para futuros pacotes publicados no npm.

9. Estrutura do projeto

A estrutura abaixo busca equilíbrio entre clareza, escalabilidade e baixa complexidade operacional. Ela é grande o suficiente para sustentar o produto, mas sem cair em overengineering.

Monorepo shape



Padrões arquiteturais recomendados

- **Monólito modular**: um produto, um repositório, módulos bem separados.
- **Ports and Adapters**: o core define contratos e os providers implementam comportamentos concretos.
- **Capability-based design**: cada adapter declara o que suporta, sem interfaces gigantes e artificiais.
- **Command / Event**: intenções e fatos separados, melhorando testabilidade e previsibilidade.
- **SOLID com pragmatismo**: usar SRP, ISP e DIP para manter o código evolutivo, sem exagero em abstrações.

10. Contratos, adapters e realtime

A interface do Studio não deve conhecer MMKV, AsyncStorage ou SQLite diretamente. Ela conhece apenas commands, events e capabilities. Isso permite adicionar novos providers no futuro sem reescrever a UI.

Exemplo de command

```
{
  "type": "key-value.set",
  "providerId": "mmkv",
  "instanceId": "default",
  "key": "user.profile",
  "value": {"name": "Caio", "premium": true}
}
```

Exemplo de event

```
{
  "type": "key-value.changed",
  "providerId": "mmkv",
  "instanceId": "default",
  "key": "user.profile",
  "source": "app",
  "timestamp": 1752860000
}
```

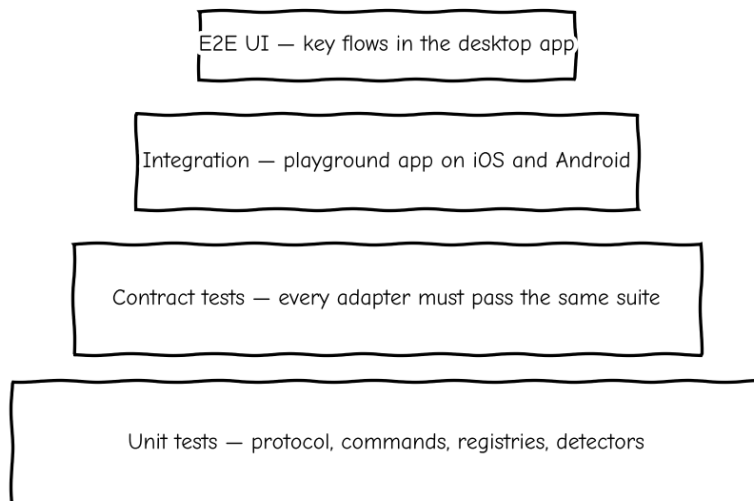
Regras importantes

- Todo provider novo deve passar por contract tests compartilhados.
- O desktop só confirma visualmente uma alteração após o runtime responder com sucesso.
- O runtime deve operar apenas em desenvolvimento.
- Quando o provider não suportar watch nativo, deve existir estratégia de instrumentação ou fallback explícito.

11. Estratégia de testes

Uma boa arquitetura aqui não vale nada sem confiança para evoluir. Como o produto será modular e baseado em adapters, a estratégia de testes é parte essencial do design.

Testing pyramid for the MVP



Goal: high confidence to add new adapters without fear.

Camadas de teste

- **Unitários:** protocol, registries, parsers, schemas Zod, detecção de dependências, serialização e handlers.
- **Contract tests:** mesma suíte validando MMKV, AsyncStorage e SQLite.
- **Integração:** app playground real com os três providers funcionando em iOS e Android.
- **E2E:** fluxos principais do desktop, como conectar, listar dados, editar, receber evento e reconectar.

Meta principal de qualidade

Adicionar um novo provider no futuro deve ser um trabalho majoritariamente local ao adapter e aos seus testes. O restante do sistema deve permanecer estável.

12. Plano de entrega sugerido

Fase	Objetivo
Fase 0 — Fundação	Definir nome de trabalho, protocolo inicial, monorepo, Tauri desktop, CLI local, runtime bootstrap
Fase 1 — AsyncStorage	Primeiro fluxo end-to-end completo: listar, editar, excluir, sincronizar com UI.
Fase 2 — MMKV	Adicionar key-value de alta qualidade com listeners e experiência muito fluida de realtime.
Fase 3 — SQLite	Adicionar browser relacional: tabelas, linhas, paginação, edição e SQL console.
Fase 4 — Hardening	Contract tests, integração iOS/Android, E2E do desktop, refinamento de UX e documentação.
Fase 5 — Alpha privada	Validar com projetos reais, medir DX e coletar feedback sobre detecção automática e tempo re

Riscos principais

- Detecção automática de instâncias pode variar entre projetos e exigir registro mínimo em casos especiais.
- Realtime no app depende da forma como cada projeto consome os dados.
- SQLite exigirá mais cuidado com paginação, performance e segurança de edição.
- A experiência de conexão local entre desktop e dispositivos físicos precisa ser extremamente bem resolvida para parecer mágica.

13. Conclusão

O MVP proposto é enxuto, mas ambicioso no que realmente importa. Em vez de tentar resolver todo o universo de DevTools de uma vez, ele ataca uma dor concreta e recorrente: inspecionar e manipular dados locais em aplicações React Native com uma experiência unificada, local, visual e em tempo real.

Por que esta proposta é forte

- Resolve um problema real de forma clara.
- Começa com um escopo controlado e com alta chance de adoção.
- Possui diferenciais fortes: zero config, 100% local e realtime.
- A arquitetura é madura o suficiente para escalar, sem overengineering.
- Cria uma base excelente para evoluir futuramente para state, network, files, logs e performance.

Frase síntese do produto

"RN Studio é o lugar onde o desenvolvedor React Native enxerga, entende e altera os dados do app sem fricção."